

Algorytm Yannakakisa

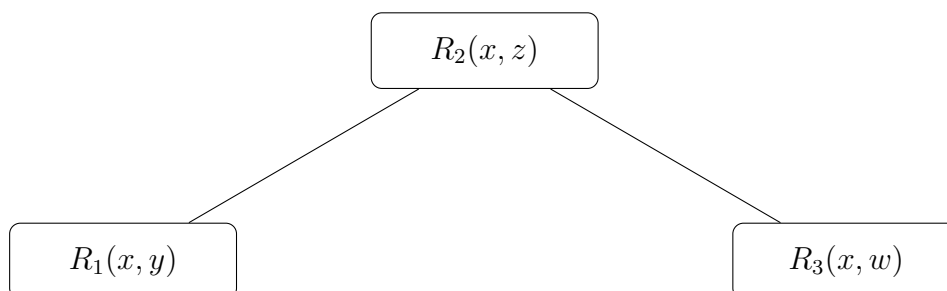
Dla uproszczenia założmy, że nasze zapytania koniunkcyjne to koniunkcje wyłącznie atomów relacyjnych (czyli postaci $R(x, y, z)$), bez stałych. Założmy również, że nie mamy kwantyfikatorów egzystencjalnych (w przypadku rozważania pełnego algorytmu Yannakakisa to założenie jest istotne!).

Drzewem złączeń (*join tree*) dla zapytania koniunkcyjnego Q nazywamy drzewo T takie, że

1. każdemu wierzchołkowi drzewa T odpowiada dokładnie jeden atom zapytania Q (to przyporządkowanie jest bijekcją),
2. dla każdej zmiennej v z zapytania Q te wierzchołki drzewa T , które odpowiadają atomom zawierającym zmienną v tworzą w T spójne poddrzewo.

Mówimy, że zapytanie Q jest *acykliczne*¹ wtw gdy posiada drzewo złączeń. Jest znany szybki algorytm (algorytm GYO) na sprawdzanie tego ale nie będziemy się tym zajmować.

Przykład drzewa złączeń dla zapytania $R_1(x, y) \wedge R_2(x, z) \wedge R_3(x, w)$:



Zauważmy, że poddrzewa dla każdej zmiennej są spójne.

- 1 (1 pkt.)** Sprawdź czy istnieje drzewo złączeń dla zapytania: $R_1(x, y) \wedge R_2(y, z) \wedge R_3(z, x)$? Jeśli tak to podaj je, jeśli nie to uzasadnij dlaczego nie. Ile w ogóle jest możliwych kandydatów na takie drzewo?

A dla zapytania $R_1(x, y) \wedge R_2(y, z) \wedge R_3(z, x) \wedge R(x, y, z)$? ²

Uproszczony algorytm Yannakakisa służy do szybkiego odfiltrowania wszystkich krotek, które na pewno nie znajdują się w wyniku. W ten sposób możemy na przykład łatwo

¹w literaturze raczej używa się nazwy α -acykliczne

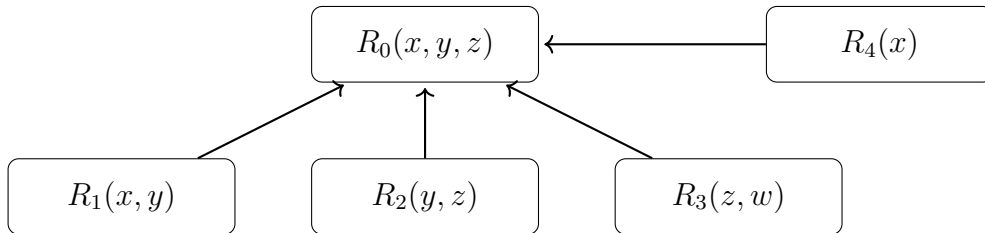
²tylko się nie zdziw :D

sprawdzić czy odpowiedź na zapytanie będzie niepusta. Jego działanie polega na przejściu od liści do korzenia wykonując w każdym wierzchołku operację semijoin z dziećmi tego wierzchołka. Semijoin definiujemy tak:

$$R \bowtie S = \{ t \in R \mid \exists s \in S, t \text{ i } s \text{ zgadza} \text{ją się na wspólnych zmiennych} \}$$

Inaczej mówiąc, semijoin odfiltrowuje z R wszystkie krotki, które nie znalazły pary w S . Zauważ, że semijoin zawsze zwraca podzbiór swojego wejścia.

Przykład ilustrujący w jaki sposób algorytm będzie przetwarzał wierzchołek z czwórką dzieci:



W wierzchołku z atomem $R_0(x, y, z)$ obliczamy

$$(((R_0(x, y, z) \bowtie R_1(x, y)) \bowtie R_2(y, z)) \bowtie R_3(z, w)) \bowtie R_4(x),$$

i możemy przechodzić wyżej.

Uproszczony algorytm Yannakakisa. Wejście: zapytanie koniunkcyjne Q , jego drzewo złączeń T i baza danych D .

1. Przypisz każdemu wierzchołkowi $n \in T$ relację (zbiór krotek) $\text{rel}(n)$ z bazy danych D odpowiadającą atomowi w wierzchołku n (nazwy kolumn przemianuj tak aby się nazywały tak jak w atomie).
 2. Przejdź drzewo bottom-up podmieniając w każdym wierzchołku n z dziećmi $n_1, \dots, n_k$, relację $\text{rel}(n)$ na $\text{rel}(n) \bowtie \text{rel}(n_1) \dots \bowtie \text{rel}(n_k)$ (tak jak w przykładzie powyżej).
- 2 (1 pkt.)** Udowodnij, że powykonaniu powyższego algorytmu dla korzenia r mamy $\text{rel}(r) \neq \emptyset$ wtw gdy zbiór odpowiedzi na Q w bazie D jest niepusty.

Uproszczony algorytm wystarcza do sprawdzania niepustości wyniku, jeśli jednak chcemy wypisać wszystkie krotki wynikowe trzeba postarać się bardziej - po fazie bottom-up musimy przekazać informację od rodziców do dzieci.

Pełny algorytm Yannakakisa (c.d.)

3. Przejdź drzewo top-down podmieniając w każdym wierzchołku n z rodzicem p , relację $\text{rel}(n)$ na $\text{rel}(n) \bowtie \text{rel}(p)$.
 4. Wykonaj finalne złączenie (join, najlepiej idąc ponownie bottom-up): $\text{rel}(n_1) \bowtie \dots \bowtie \text{rel}(n_k)$, dla wszystkich wierzchołków $n_1, \dots, n_k$ drzewa T i zwróć wynik. Każda krotka powstająca podczas tego złączenia będzie częścią wyniku (złączenia po drodze nie wygenerują żadnej krotki, która potem zostałaby wyeliminowana).
- 3 (1 pkt.)** Jaka jest złożoność uproszczonego oraz pełnego algorytmu Yannakakisa? A jaka jest złożoność algorytmu *brute-force* czyli wykonania pełnego złączenia wszystkich relacji bez filtrowania?

Datalog

- 4 (1 pkt.)** Rozważmy następujące zapytanie w Datalogu.

$$\begin{aligned} T(X, Y) &:- E(X, Y). \\ T(X, Y) &:- T(X, Z), T(Z, Y). \end{aligned}$$

Przypomnij definicję semantyki dla Datalogu, a następnie pokaż, że dla każdego $i \in \mathbb{N}_+$ zachodzi $T^i = \{(a, b) \mid \text{istnieje ścieżka z } a \text{ do } b \text{ o długości } \leq 2^{i-1}\}$

Napisz następujące zapytania datalogowe. Użyj stałych n i m tam gdzie jest to potrzebne.

- 5 (0.5 pkt.)** Zwróć wierzchołki, do których można dojść ścieżką z n lub ścieżką z m .
- 6 (0.5 pkt.)** Zwróć wierzchołki, do których można dojść ścieżką z n i ścieżką z m .
- 7 (1 pkt.)** Zwróć pary wierzchołków, do których można dojść z wierzchołka n ścieżkami o tej samej długości.
- 8 (1 pkt.)** Zwróć pary wierzchołków x, y , takie, że z wierzchołka n do x oraz do y można dojść ścieżkami, które mają różną długość.

Rozważamy grafy z jedną relacją ternarną $E(s, t, a)$. W trzeciej kolumnie pamiętamy kolor krawędzi (obsługujemy paletę 16 milionów kolorów). Napisz następujące zapytania datalogowe lub udowodnij, że dane zapytanie nie istnieje.

- 9 (0.5 pkt.)** Zwróć pary wierzchołków x, y takie, że nie istnieje ścieżka z x do y .
- 10 (0.5 pkt.)** Zwróć pary wierzchołków x, y , takie, że z x do y można przejść jednokolorową ścieżką.

11 (0.5 pkt.) Zwróć pary wierzchołków x, y takie, że każda ścieżka z x do y składa się z krawędzi co najmniej dwóch kolorów.

12 (1 pkt.) Zwróć pary wierzchołków x, y , takie, że z x do y można przejść ścieżką składającą się z krawędzi nie więcej niż dwóch kolorów.